



DOCUMENTO DE ARQUITECTURA DEL PROYECTO

Sistema Clínica Pro - Módulo de Gestión de Citas
Clínica Naturista Quantum

Grupo No. 2

NOMBRE	CARNÉ
Christian Adrean Ordoñez Xiloj	3590-23-5687
Ricardo Enmanuel Revolorio Pérez	3590-23-8356
Allan Ricardo Quiej Ixmata	3590-17-3156
Sergio Francisco Jiménez Lucas	3590-23-8632
Rodrigo Vidal Guzmán Díaz	3590-23-6197

Contenido

- 1. INFORMACIÓN GENERAL Y CONTINUIDAD DEL PROYECTO 3
 - 1.1 Alcance funcional y actualización tecnológica 3
 - 1.2 Evidencia del proceso actual documentado en el DERCAS 4
- 2. USUARIOS, ROLES Y PERMISOS 5
- 3. PROCESO PRINCIPAL: SOLICITUD Y PROGRAMACIÓN DE UNA CITA..... 6
 - 3.1 Estados y reglas de negocio críticas 7
- 4. MÓDULOS DEL SISTEMA Y VISTA LÓGICA 8
- 5. DISEÑO Y JUSTIFICACIÓN DE LA ARQUITECTURA..... 9
- 6. VISTA DE DESARROLLO 11
- 7. VISTA DE PROCESOS..... 12
- 8. VISTA FÍSICA O DE DESPLIEGUE..... 14
- 9. APLICACIÓN DE MVC, CAPAS Y TECNOLOGÍAS 15
 - 9.1 MVC adaptado a Blazor..... 15
 - 9.2 Tecnologías propuestas 15
- 10. RIESGOS TÉCNICOS Y MEDIDAS PREVENTIVAS..... 16
- 11. ORGANIZACIÓN DEL EQUIPO Y EVIDENCIA 17
- 12. ENTREGABLES Y CONCLUSIÓN..... 18
 - 12.1 Conclusion..... 18

1. INFORMACIÓN GENERAL Y CONTINUIDAD DEL PROYECTO

Clínica Pro continúa el proyecto desarrollado en Análisis de Sistemas I y toma como base funcional el DERCAS del Módulo de Gestión de Citas. La arquitectura actual conserva el proceso real, los actores, las reglas de negocio y el alcance aprobado, pero sustituye la tecnología histórica de PHP/MySQL por una solución moderna basada en .NET 8 y SQL Server.

Elemento	Definición
Nombre del sistema	Clínica Pro - Módulo de Gestión de Citas.
Organización	Clínica Naturista Quantum.
Problema	La clínica gestiona citas mediante agenda física, llamadas, mensajes y búsqueda manual de expedientes, generando demoras, riesgo de doble reserva, poca trazabilidad y tareas administrativas repetitivas.
Objetivo general	Implementar un sistema web que permita solicitar, revisar, programar y dar seguimiento a citas médicas, con validación de disponibilidad, control de estados, notificaciones, seguridad por roles y trazabilidad.

1.1 Alcance funcional y actualización tecnológica

Incluido en la versión 1	Fuera de alcance
Autenticación y roles; pacientes básicos; médicos, especialidades y horarios; solicitudes de cita; confirmación por secretaría; reprogramación y cancelación; registro de llegada; atención; no presentación; historial; notificaciones; reportes y auditoría.	Triaje; consulta clínica detallada; diagnósticos; signos vitales; recetas completas; producción naturista; farmacia; inventario; pagos y facturación; compras y proveedores.

Decisión tecnológica vigente: el DERCAS original documentaba PHP, Apache y MySQL. Para Análisis de Sistemas II se adopta Blazor WebAssembly, ASP.NET Core Web API .NET 8, Entity Framework Core 8 y SQL Server. El cambio tecnológico no altera las reglas funcionales aprobadas; las reconstruye con una arquitectura mantenible, verificable y preparada para pruebas.

1.2 Evidencia del proceso actual documentado en el DERCAS

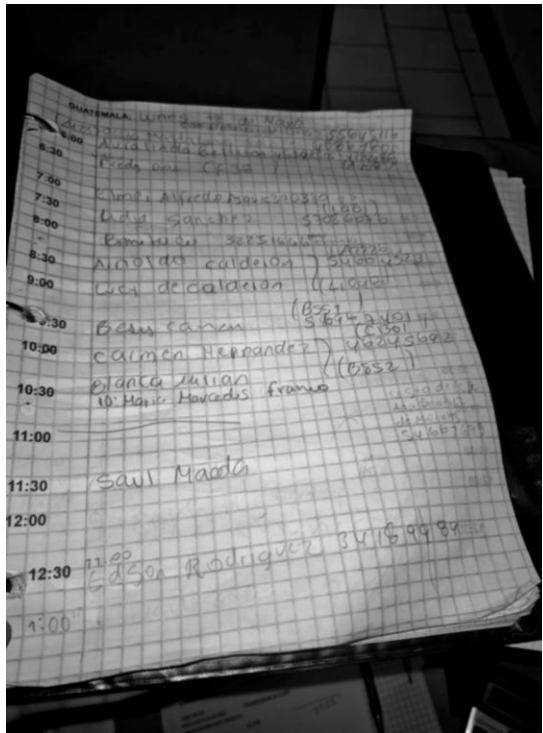


Figura 1. Agenda física utilizada en el proceso AS-IS (datos difuminados).

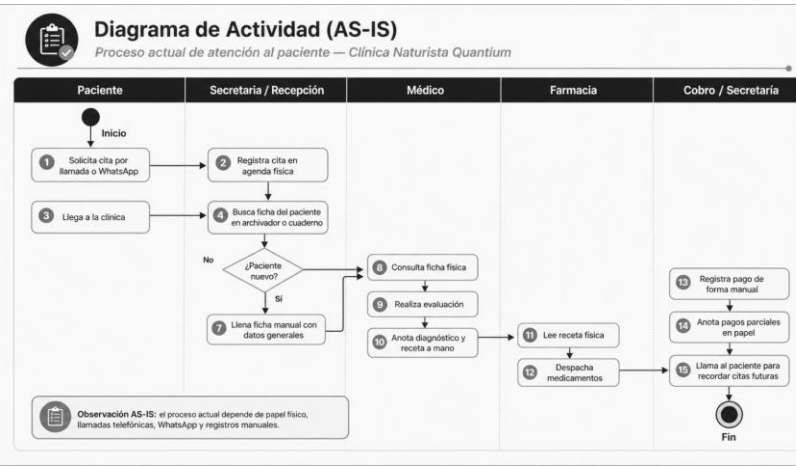


Figura 2. Diagrama AS-IS del levantamiento de requerimientos.

Las evidencias anteriores justifican que el proyecto se enfoque en un proceso completo y no en formularios CRUD aislados: la cita debe recorrer solicitud, revisión, autorización, ejecución, registro y reporte.

2. USUARIOS, ROLES Y PERMISOS

Rol	Responsabilidad	Acciones principales	Restricciones
Paciente	Autogestión de sus citas.	Registrarse, actualizar perfil, seleccionar especialidad, solicitar horario, consultar estado, confirmar asistencia y cancelar según reglas.	No programa directamente la cita ni accede a datos de otros pacientes.
Secretaria	Administrar la agenda clínica.	Revisar solicitudes, confirmar o rechazar, crear pacientes, reprogramar, cancelar, registrar llegada y marcar no presentado.	No marca una cita como atendida ni modifica reglas de seguridad.
Médico	Gestionar su agenda asignada.	Consultar agenda, ver datos básicos del paciente y marcar atención finalizada.	Solo accede a sus citas y no confirma solicitudes administrativas.
Administrador	Administrar configuración y seguridad.	Gestionar usuarios, roles, médicos, especialidades, horarios, parámetros, auditoría y autorizaciones excepcionales.	Las acciones sensibles quedan auditadas.

3. PROCESO PRINCIPAL: SOLICITUD Y PROGRAMACIÓN DE UNA CITA

El paciente no crea una cita Programada de forma inmediata. Primero envía una solicitud que reserva provisionalmente el horario. La secretaria realiza la revisión administrativa y, al aprobarla, la solicitud pasa a Programada. Esta decisión reduce errores y mantiene control operativo sobre la agenda.

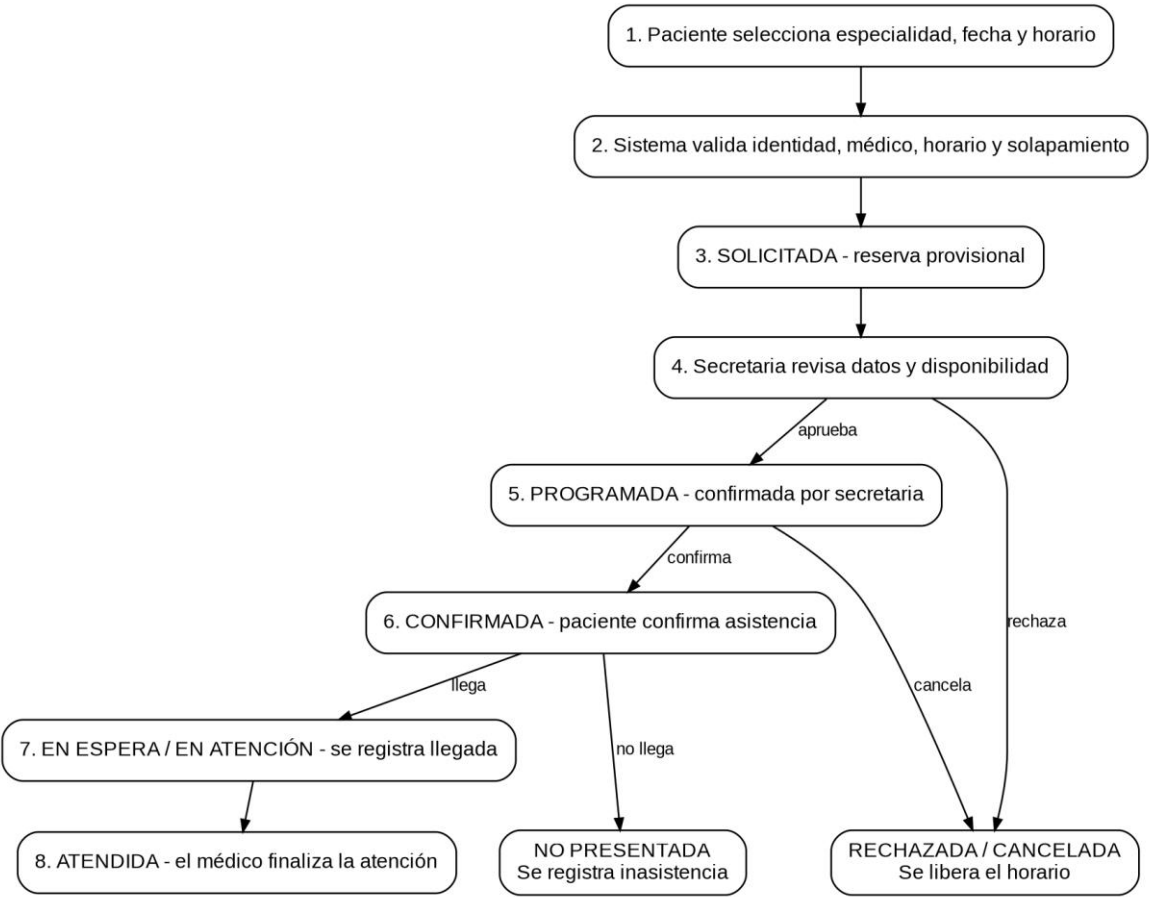


Figura 3. Flujo principal actualizado de solicitud, confirmación administrativa y atención.

Etapa	Responsabilidad y resultado
1. Solicitud	El paciente inicia sesión, selecciona especialidad y consulta horarios disponibles.
2. Asociación médica	El sistema identifica al médico primario activo asociado con la especialidad y utiliza sus horarios.
3. Validación	El backend valida identidad, fecha futura, horario laboral, médico asignado y ausencia de solapamiento.
4. Registro	La solicitud se almacena en estado Solicitada y bloquea provisionalmente el horario.
5. Revisión	La secretaria verifica datos, motivo, disponibilidad y coherencia operativa.
6. Confirmación	Al aprobar, el estado cambia a Programada, se registra la responsable y se notifica al paciente.
7. Seguimiento	El paciente confirma asistencia; la secretaria registra llegada; el médico marca Atendida o se registra No presentada.
8. Reporte	El historial conserva las transiciones y alimenta reportes de agenda, cancelaciones, reprogramaciones e inasistencia.

3.1 Estados y reglas de negocio críticas

Estado	Significado	Transición autorizada
Solicitada	Pendiente de revisión; reserva provisional.	Secretaria confirma, rechaza o reprograma.
Programada	Solicitud aprobada administrativamente.	Paciente confirma, cancela o secretaria reprograma.
Confirmada	El paciente manifestó que asistirá.	Secretaria registra llegada.
En espera / En atención	El paciente llegó y está siendo atendido.	Médico marca Atendida.
Atendida / Cancelada / No presentada / Rechazada	Estados finales; no se eliminan.	Solo se consultan en historial y reportes.

- Una especialidad debe tener un único médico primario activo en la fase 1; el modelo admite más médicos en el futuro mediante Médico-Especialidad.
- Las solicitudes en estado Solicitada participan en la validación de disponibilidad para impedir reservas duplicadas.
- La confirmación de secretaria revalida el horario dentro de una transacción antes de cambiar a Programada.
- No se permite una cita en el pasado, fuera del horario del médico o superpuesta con otra cita activa.
- La tercera reprogramación requiere autorización del administrador y toda reprogramación registra fecha anterior, nueva fecha, motivo y usuario.
- Una cancelación con menos de dos horas se registra como No presentada.
- Solo el médico asignado puede marcar la cita como Atendida.

4. MÓDULOS DEL SISTEMA Y VISTA LÓGICA

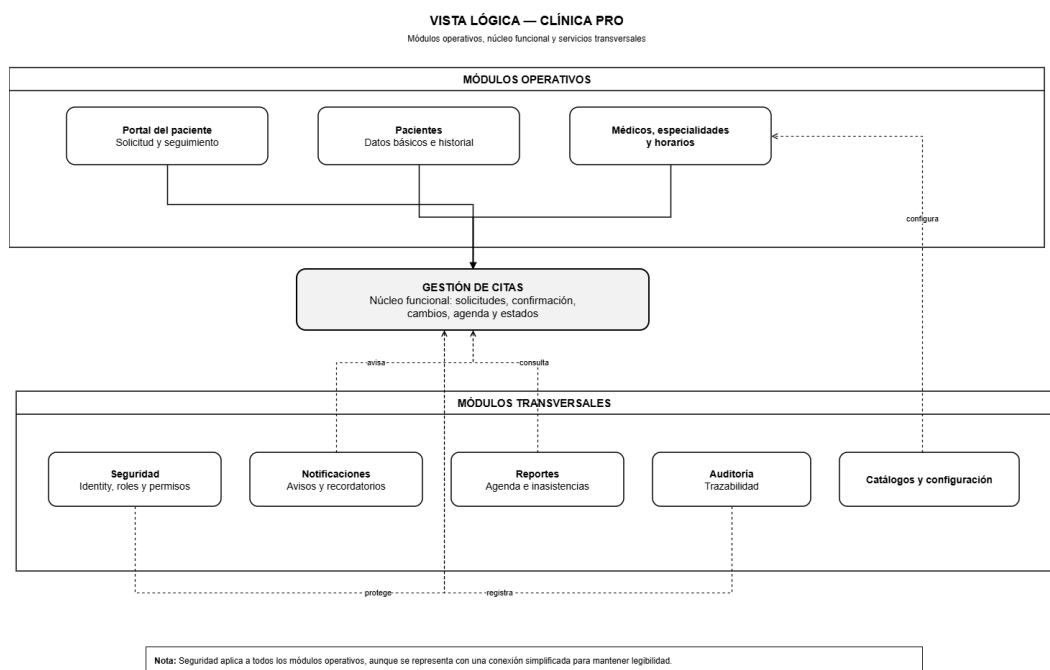
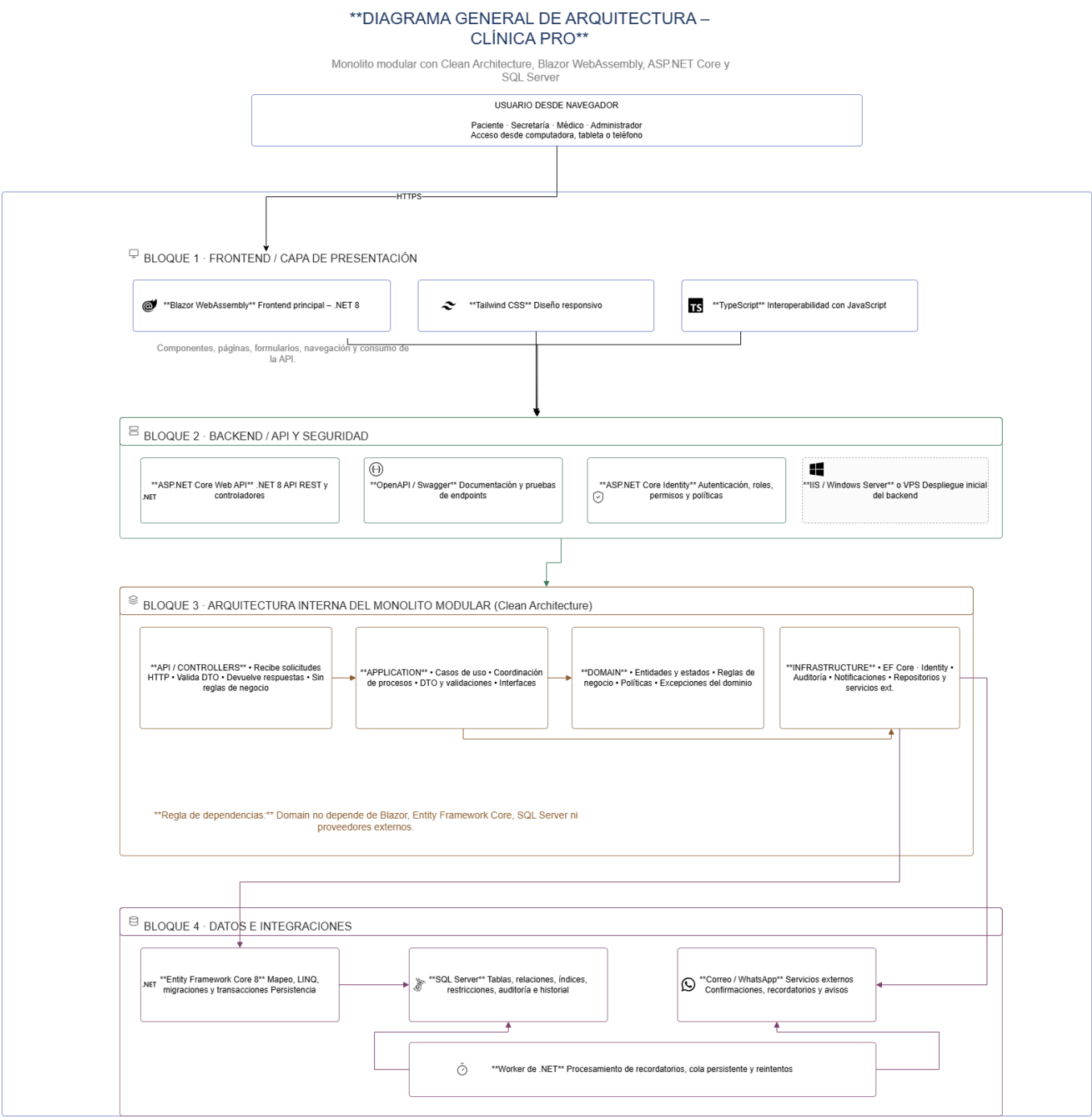


Figura 4. Vista lógica de módulos operativos y transversales de Clínica Pro.

Módulo	Tipo	Responsabilidad	Información administrada / comunicación
Portal del paciente	Operativo	Registro, solicitud y seguimiento de estados.	Perfil, solicitud, confirmación y cancelación; se comunica con Seguridad, Citas y Notificaciones.
Pacientes	Operativo	Datos básicos, contacto e historial de citas.	Paciente, documento, contacto e historial; se comunica con Citas, Seguridad y Reportes.
Médicos y horarios	Operativo	Médicos, especialidades, relación médico-especialidad y disponibilidad.	Médico, especialidad y horario; se comunica con Citas, Catálogos y Administración.
Gestión de citas	Núcleo	Solicitudes, confirmación, agenda, cambios y estados.	Cita e historial; se comunica con todos los módulos.
Seguridad	Transversal	Autenticación, roles, permisos y bloqueo de acceso.	Usuario, rol y políticas; protege Frontend, API y Auditoría.
Notificaciones	Transversal	Confirmaciones, recordatorios y avisos de cambio.	Notificación e intento; se comunica con Citas y proveedores externos.
Reportes	Transversal	Agenda, citas por estado, médico e inasistencias.	Consulta Citas, Pacientes y Médicos.
Auditoría y catálogos	Transversal	Trazabilidad, especialidades, estados y parámetros.	Auditoría, Configuración y catálogos; se comunica con todos los módulos.

Entidades centrales: Usuario, Rol, Paciente, Médico, Especialidad, MédicoEspecialidad, HorarioMédico, Cita, HistorialCita, Notificación, IntentoNotificación, Auditoría y Configuración. No se crearán entidades de triaje, producción, lotes, materias primas, inventario o despacho de medicamentos.

5. DISEÑO Y JUSTIFICACIÓN DE LA ARQUITECTURA



Arquitectura seleccionada: aplicación web cliente-servidor implementada como monolito modular, organizada con Clean Architecture, arquitectura en capas y MVC adaptado. El frontend Blazor consume una API REST de ASP.NET Core; las reglas se ejecutan en el servidor y SQL Server no es accesible directamente desde el navegador.

Figura 5. Diagrama general de arquitectura y recorrido de una petición.

Criterio	Justificación
Mantenibilidad	Presentación, casos de uso, dominio e infraestructura están separados. Los cambios visuales no alteran las reglas de citas y el acceso a datos puede sustituirse sin modificar el dominio.

Criterio	Justificación
Escalabilidad	Los módulos tienen límites definidos; la API puede escalarse horizontalmente y los servicios externos se reemplazan mediante interfaces.
Facilidad de trabajo	Una sola solución .NET y un despliegue inicial reducen complejidad. El equipo puede distribuirse por módulos y capas usando Pull Requests.
Seguridad	El cliente solo consume endpoints HTTPS. Identity, políticas de autorización y validaciones del servidor protegen datos y acciones.
Viabilidad	El monolito modular evita la complejidad de microservicios, pero conserva una estructura preparada para separar módulos cuando el volumen lo justifique.
Riesgos o dificultades	Curva de aprendizaje de Blazor/Tailwind, migración de reglas desde PHP/MySQL, control de concurrencia y operación de notificaciones externas.

Regla de dependencias: Domain no dependerá de Blazor, Entity Framework Core, SQL Server ni proveedores externos. Application definirá los casos de uso; Infrastructure implementará persistencia, identidad, auditoría y notificaciones; Api expondrá controladores; Client contendrá páginas y componentes Blazor.

6. VISTA DE DESARROLLO

La solución se organizará por proyectos de capa y carpetas por funcionalidad. La estructura evita un proyecto excesivamente fragmentado y permite que cada integrante trabaje en una funcionalidad completa sin mezclar responsabilidades.

ClinicaPro/	
src/	
ClinicaPro.Client/	Blazor, Tailwind y TypeScript
ClinicaPro.Api/	Controladores, middleware y OpenAPI
ClinicaPro.Application/	Casos de uso, DTO y validaciones
ClinicaPro.Domain/	Entidades, estados y reglas
ClinicaPro.Infrastructure/	EF Core, Identity, auditoría y avisos
ClinicaPro.Contracts/	Contratos compartidos de la API
tests/	UnitTests, IntegrationTests y EndToEndTests
database/	Scripts y documentación de migración
docs/	Arquitectura y diagramas editables
deploy/	Configuración de publicación
.github/workflows/	Compilación y pruebas automáticas
ClinicaPro.sln	

Proyecto	Contenido	Dependencias permitidas
Client	Componentes, páginas, formularios, navegación y servicios HTTP.	Contracts; nunca Infrastructure ni SQL Server.
Api	Endpoints, autenticación de solicitudes, middleware y manejo de errores.	Application y Contracts.
Application	Casos de uso: solicitar, confirmar, reprogramar, cancelar y finalizar.	Domain e interfaces.
Domain	Entidades, estados, políticas y excepciones del negocio.	Ninguna dependencia tecnológica.
Infrastructure	EF Core, SQL Server, Identity, notificaciones, archivos y auditoría.	Application y Domain.
Tests	Pruebas unitarias, integración y recorridos completos.	Proyectos bajo prueba.

- Ramas main, develop, feature/*, fix/* y docs/*; no se trabaja directamente en main.
- Cada funcionalidad se integra mediante Pull Request, revisión de otro integrante y pruebas exitosas.
- Secretos, cadenas de conexión, respaldos y datos de pacientes no se almacenan en GitHub.

7. VISTA DE PROCESOS

La vista representa el proceso crítico en dos momentos: creación de la solicitud por el paciente y confirmación administrativa por la secretaria. La disponibilidad se valida dos veces para evitar condiciones de carrera: al solicitar y al confirmar.

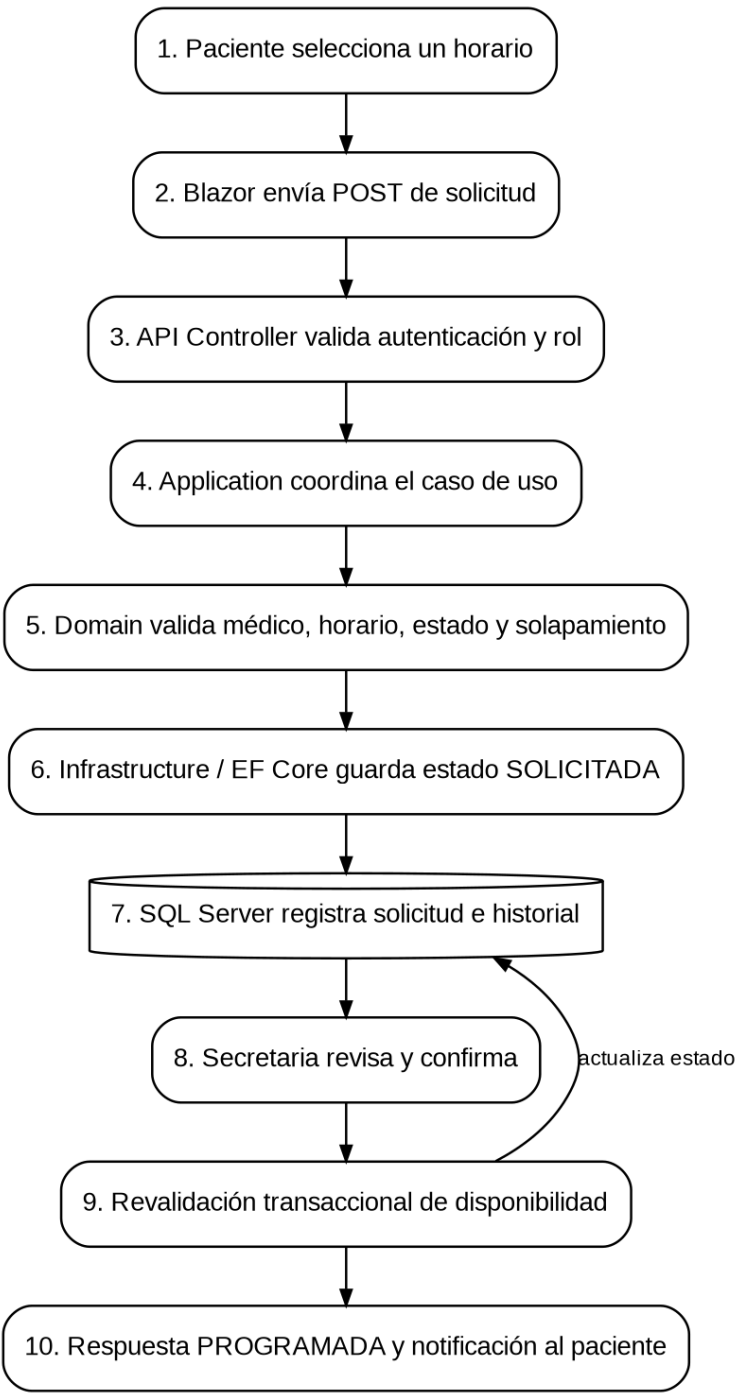


Figura 6. Secuencia de solicitud, revalidación y confirmación de una cita.

Validación	Capa responsable	Respuesta
Usuario y permisos	API / Identity	Rechaza usuarios no autenticados o roles sin autorización.

Validación	Capa responsable	Respuesta
Especialidad y médico primario	Application / Domain	No permite solicitar si no existe una asociación activa y única.
Horario laboral	Domain	El horario debe pertenecer a la agenda configurada del médico.
Solapamiento	Domain + SQL Server	Solicitudes y citas activas bloquean intervalos coincidentes.
Estado válido	Domain	Solo Solicitada puede pasar a Programada mediante secretaría.
Trazabilidad	Application / Infrastructure	Cada cambio registra usuario, fecha, estado anterior, nuevo estado y motivo.

Resultado del proceso: al concluir la confirmación, SQL Server conserva la cita en estado Programada, el historial de transición, la secretaria responsable y la notificación pendiente. Si la secretaria rechaza, el sistema registra el motivo, libera el horario y notifica al paciente.

Control de concurrencia: la confirmación utilizará una transacción y una restricción o índice de apoyo para impedir que dos solicitudes terminen programadas en el mismo intervalo del mismo médico.

8. VISTA FÍSICA O DE DESPLIEGUE

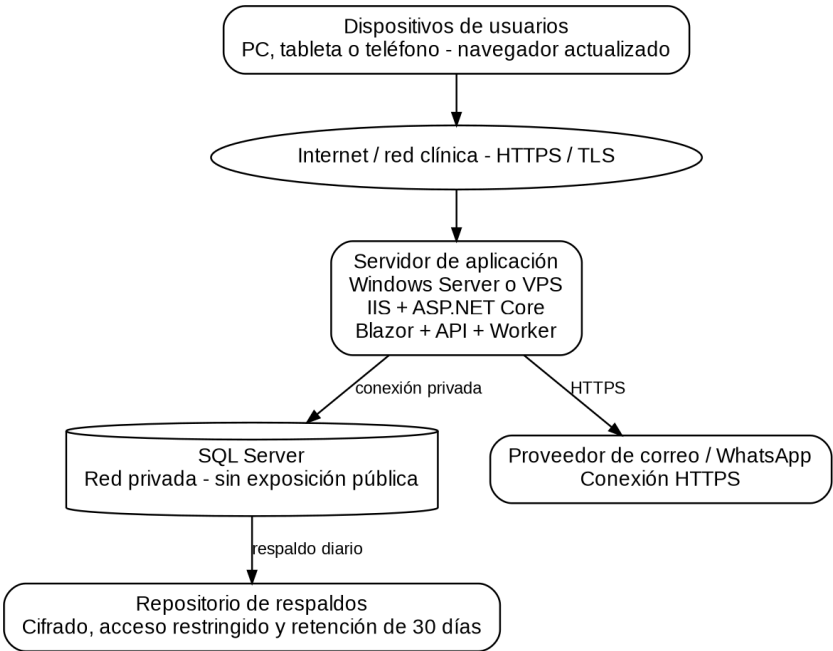


Figura 7. Vista de despliegue propuesta para la primera versión.

Aspecto	Definición
Tipo de aplicación	Aplicación web responsiva; acceso desde navegador en computadora, tableta o teléfono.
Frontend y API	Blazor y ASP.NET Core se publican inicialmente en el mismo servidor y dominio.
Servidor	Windows Server o VPS con IIS y ASP.NET Core Hosting Bundle.
Base de datos	SQL Server en red privada; no expuesto a Internet ni accesible desde el navegador.
Comunicación	HTTPS obligatorio; la API intercambia JSON y valida autenticación, rol y permisos.
Procesos automáticos	Worker de .NET procesa recordatorios y reintentos con una cola persistente en SQL Server.
Respaldos	Respaldo diario de SQL Server, retención inicial de 30 días, copia cifrada y prueba periódica de restauración.

- ASP.NET Core Identity para autenticación, bloqueo por intentos fallidos y recuperación de acceso.
- Autorización por roles y políticas; ocultar botones en Blazor no sustituye la validación de la API.
- Variables de entorno o almacén de secretos para cadenas de conexión y credenciales.
- Logs técnicos sin diagnósticos ni información clínica sensible; auditoría funcional separada.
- Ambientes Development, Testing y Production con configuraciones y bases distintas.

9. APLICACIÓN DE MVC, CAPAS Y TECNOLOGÍAS

9.1 MVC adaptado a Blazor

Responsabilidad	Implementación	Ejemplo en Clínica Pro
Vista o presentación	Componentes y páginas Blazor con Tailwind CSS.	Formulario para solicitar cita y panel de solicitudes pendientes.
Controlador	Controladores de ASP.NET Core Web API.	AppointmentsController recibe una solicitud o confirmación.
Lógica o servicios	Application y Domain.	AppointmentService valida médico, estados, disponibilidad y permisos.
Acceso a datos	Infrastructure con Entity Framework Core.	DbContext consulta y actualiza citas dentro de una transacción.
Base de datos	SQL Server.	Tablas, relaciones, índices, restricciones e historial.

Ejemplo completo: el paciente envía una solicitud desde Blazor; el controlador recibe el DTO; Application coordina el caso de uso; Domain valida especialidad, médico y horario; Infrastructure guarda la solicitud; la secretaria confirma mediante otro endpoint y el sistema cambia el estado a Programada.

9.2 Tecnologías propuestas

Componente	Tecnología propuesta	Justificación
Frontend	Blazor WebAssembly en .NET 8	Interfaz rica en C#, separación clara del backend y reutilización de contratos.
Estilos	Tailwind CSS	Diseño responsivo y mantenible mediante clases utilitarias.
Interoperabilidad	TypeScript	Calendario, funciones del navegador e integración JavaScript; sin reglas clínicas.
Backend	ASP.NET Core Web API .NET 8	Controladores, middleware, autenticación, inyección de dependencias y soporte de pruebas.
Persistencia	Entity Framework Core 8	Mapeo de entidades, migraciones, transacciones y consultas LINQ.
Base de datos	SQL Server	Integridad relacional, concurrencia, respaldos y herramientas administrativas.
Seguridad	ASP.NET Core Identity	Usuarios, roles, bloqueo, recuperación de contraseña y políticas.
Documentación API	OpenAPI / Swagger	Contrato visible y pruebas de endpoints.
Pruebas	xUnit, bUnit y Playwright	Reglas, componentes y recorridos completos.
Repositorio	GitHub + GitHub Actions	Control de versiones, evidencia de commits, revisión y compilación automática.
Diagramas	Draw.io y archivos DOT	Diagramas legibles, versionables y editables.
Despliegue	IIS en Windows Server / VPS	Opción viable para la primera versión y compatible con ASP.NET Core.

10. RIESGOS TÉCNICOS Y MEDIDAS PREVENTIVAS

Riesgo	Impacto	Medida preventiva
Migración desde PHP/MySQL	Pérdida o interpretación incorrecta de datos y reglas.	Migración por etapas, diccionario de datos, scripts repetibles y conciliación de registros.
Doble reserva por concurrencia	Dos solicitudes podrían ocupar el mismo horario.	Revalidación transaccional, bloqueo lógico e índices o restricciones de apoyo.
Fallo de correo o WhatsApp	El paciente no recibe avisos.	Cola persistente, reintentos, registro de errores y canal alternativo.
Curva de aprendizaje Blazor/Tailwind	Retrasos y código inconsistente.	Prototipo técnico temprano, guía de estilos, revisión y programación en pares.
Crecimiento del alcance	No completar el módulo principal.	Lista formal de exclusiones y control de cambios.
Seguridad y secretos	Exposición de datos o credenciales.	Identity, políticas, HTTPS, secretos fuera de GitHub y revisión de permisos.
Actualización tecnológica	Dependencias incompatibles o versión sin mantenimiento.	Revisar el ciclo de soporte y mantener un plan de actualización controlada.

11. ORGANIZACIÓN DEL EQUIPO Y EVIDENCIA

Integrante	Rol	Responsabilidad	Evidencia esperada
Sergio Francisco Jiménez Lucas	Coordinador / arquitectura	Integrar arquitectura, casos de uso y backend base.	Diagrama general, ADR, Pull Requests y commits.
Allan Ricardo Quiej Ixmata	Analista funcional / QA	Actualizar reglas, criterios y pruebas del proceso.	Matriz de reglas, casos de prueba y evidencias.
Ricardo Enmanuel Revolorio Pérez	Frontend Blazor / UX	Portal del paciente y panel de secretaría.	Componentes, capturas, pruebas bUnit y commits.
Christian Adrean Ordoñez Xiloj	Datos / infraestructura	Modelo SQL Server, EF Core y migración.	Diagrama de datos, migraciones y pruebas de integración.
Rodrigo Vidal Guzmán Díaz	Seguridad / DevOps / documentación	Identity, GitHub Actions, despliegue y documento final.	Políticas, workflow, guía de despliegue y commits.

12. ENTREGABLES Y CONCLUSIÓN

Elemento	Definición / estado
Repositorio GitHub	https://github.com/Christian-Bo/Clinica_Quantium.git
Rama de entrega	main, integrada mediante Pull Requests aprobados.
Diagramas editables	https://drive.google.com/drive/folders/1SsqUN8RjnW-9AMNHfIPTcrHGmwznzWfAY?usp=sharing
Número de grupo	2

12.1 Conclusion

La propuesta transforma el prototipo y los requerimientos previos en un sistema funcional centrado en el proceso real de citas. El paciente solicita un horario según especialidad; el sistema reserva provisionalmente y valida disponibilidad; la secretaria confirma y convierte la solicitud en una cita Programada; y las capas de dominio, aplicación e infraestructura garantizan reglas, seguridad, persistencia y trazabilidad. El monolito modular reduce el riesgo de implementación, mientras que Clean Architecture permite mantener y ampliar Clínica Pro sin incorporar en esta fase módulos fuera de alcance.